



**School of
Arts and Sciences**

CSC464 – Deep Learning & Natural Language Processing

Instructor: Dr. Ahmad Audi

Hate Speech & Toxicity Detection

Tia Aoun

Alaeddine Fathallah

Ali Mourad

1. Introduction

The rapid growth of online communication platforms such as social media, discussion forums, and online gaming communities has significantly increased the amount of user-generated content shared daily. While these platforms enable communication and interaction between users worldwide, they also face major challenges related to toxic and abusive language. Harmful comments such as insults, hate speech, threats, and offensive language can negatively affect users and create unsafe online environments.

Due to the extremely large volume of online content, manually reviewing and moderating comments has become difficult, time-consuming, and expensive. As a result, automatic toxicity detection systems have become increasingly important. These systems use Natural Language Processing (NLP), machine learning, and deep learning techniques to analyze textual content and classify comments according to their toxicity level.

This project focuses on detecting toxic comments using the Jigsaw Toxic Comment Classification dataset. Different machine learning and deep learning models were implemented and compared in order to evaluate their effectiveness for text classification tasks. The project includes the complete pipeline of loading and preprocessing the dataset, feature engineering, model training, evaluation, and performance comparison.

Four models were implemented in this project:

- Logistic Regression
- Linear Support Vector Machine (SVM)
- Long Short-Term Memory (LSTM)
- Bidirectional Long Short-Term Memory (BiLSTM)

The main objective of this work is to compare traditional machine learning approaches with deep learning approaches for toxicity detection while analyzing their strengths, weaknesses, and overall classification performance.

2. Project Objectives

The main objective of this project is to develop an automated system capable of classifying online comments according to their toxicity level. The system aims to identify harmful comments and distinguish them from normal non-toxic comments using machine learning and deep learning techniques.

The specific objectives of the project are:

- To preprocess and clean textual data for toxicity classification
- To convert the original multi-label dataset into a simplified three-class classification problem
- To implement and compare multiple machine learning and deep learning models
- To evaluate model performance using different evaluation metrics
- To analyze the strengths and limitations of each model
- To determine which approach performs best for toxic comment classification

The project also aims to demonstrate the importance of preprocessing, feature engineering, and class balancing in Natural Language Processing tasks.

3. Dataset Description

The project uses the Jigsaw Toxic Comment Classification Challenge dataset available on Kaggle. The dataset contains comments collected from Wikipedia discussion pages and includes labels representing different forms of toxic behavior.

	A	B	C	D	E	F	G	H
1	id	comment_text	toxic	severe_toxic	obscene	threat	insult	identity_hate
2	000997932d777bf	Explanation	0	0	0	0	0	0
3	000103f0d9c6b60f	D'aww! He matches this b	0	0	0	0	0	0
4	000113f07ec002fd	Hey man, I'm really not tryi	0	0	0	0	0	0
5	0001b41b1c6bb37e	*	0	0	0	0	0	0
6	0001d958c54c6e35	You, sir, are my hero. Any c	0	0	0	0	0	0
7	00025465d4725e87	*	0	0	0	0	0	0
8	0002bcb3da6cb337	COCKSUCKER BEFORE YO	1	1	0	1	0	0
9	00031b1e95af7921	Your vandalism to the Matt	0	0	0	0	0	0
10	00037261f536c51d	Sorry if the word 'nonsense	0	0	0	0	0	0
11	00040093b2687caa	alignment on this subject a	0	0	0	0	0	0
12	000530008490edc	*	0	0	0	0	0	0
13	000545e18b50dd4	bbq	0	0	0	0	0	0
14	0005c987bdfc9d4b	Hey... what is it..	1	0	0	0	0	0
15	0006f16e4e9f292e	Before you start throwing	0	0	0	0	0	0
16	00070ef96486d6f9	Oh, and the girl above start	0	0	0	0	0	0
17	00078f8ce7eb276d	*	0	0	0	0	0	0
18	0007e25b2121310b	Bye!	1	0	0	0	0	0
19	000897889268bc93	REDIRECT Talk:Voydan Pop	0	0	0	0	0	0
20	0009801bd85e5806	The Mitsurugi point made n	0	0	0	0	0	0
21	0009eaea3325de8c	Don't mean to bother you	0	0	0	0	0	0
22	000b08c464718505	*	0	0	0	0	0	0
23	000b0867774845	*	0	0	0	0	0	0
24	000c0df0995809fa	*	0	0	0	0	0	0
25	000c6a3f0cd3ba8e	*	0	0	0	0	0	0
26	000cfee90f50d471	*	0	0	0	0	0	0
27	000eefc67a2c930f	Radial symmetry	0	0	0	0	0	0
28	000f35deef84dc4a	There's no need to apologiz	0	0	0	0	0	0
29	000fab30195c5e1	Yes, because the mother o	0	0	0	0	0	0
30	0010307a3a50a353	*	0	0	0	0	0	0

Figure 1. Sample Records from the Jigsaw Toxic Comment Classification Dataset

The original dataset contains the following six binary labels:

- toxic
- severe_toxic
- obscene
- threat
- insult
- identity_hate

Since the original dataset is multi-label, it was transformed into a simpler three-class classification problem in order to make the task easier to analyze and compare across different models. The mapping rules used in the project are shown below.

Final Class	Mapping Rule
Abusive	severe_toxic OR threat OR insult OR identity_hate
Toxic	toxic OR obscene (if not abusive)
Non-toxic	none of the labels are active

Figure 2. Mapping Rules Used to Convert the Original Multi-Label Dataset into Three Final Classes

The final dataset therefore contains three mutually exclusive classes:

- Non-toxic
- Toxic
- Abusive

Final Label	Example Comment
0 (Non-toxic)	“I really like this video”
1 (Toxic)	“This is so stupid”
2 (Abusive)	“You’re useless, go away”

Figure 3. Examples of Comments for Each Final Class

After creating the new labels, the class distribution was analyzed using visualization techniques.

The results showed that the dataset is imbalanced, with non-toxic comments representing the majority of the samples.

Because of this imbalance, evaluation metrics such as macro F1-score were considered important in addition to accuracy in order to fairly evaluate the models.

4. Data Preprocessing

Text preprocessing is an essential step in Natural Language Processing tasks because raw textual data often contains noise, inconsistencies, and irrelevant information that may negatively affect model performance. In this project, several preprocessing operations were applied to clean and normalize the comments before training the models.

The preprocessing pipeline included the following steps:

1. Converting all text to lowercase
2. Removing URLs and web links
3. Removing user mentions and HTML tags
4. Removing numbers and punctuation
5. Tokenizing the text into individual words
6. Removing English stopwords
7. Applying lemmatization to reduce words to their root forms

The preprocessing functions were implemented using Python libraries such as NLTK and regular expressions.

Lowercasing was used to ensure consistency between words such as “Hello” and “hello”. URLs, HTML tags, and user mentions were removed because they do not contribute meaningful semantic information for toxicity classification. Punctuation and numerical values were also removed to reduce unnecessary noise in the dataset.

After cleaning the text, tokenization was applied to split comments into individual tokens.

Stopword removal was then performed to eliminate common English words such as “the”, “is”, and “and”, which usually carry little semantic meaning. Finally, lemmatization was applied to reduce words to their base forms, helping reduce vocabulary size and improve generalization.

An example of the preprocessing process is shown below:

Original Text	Processed Text
“Check this out @user!!! https://x.com ”	“check visit”

Figure 4. Example of Text Preprocessing Applied to a Toxic Comment

After preprocessing, empty comments were removed from the dataset before proceeding to model training.

5. Feature Engineering

Feature engineering is an important step in Natural Language Processing because machine learning models cannot directly process raw text data. Therefore, textual information must first be converted into numerical representations that models can understand and analyze. In this project, different feature engineering techniques were used depending on the type of model being trained.

Traditional machine learning models such as Logistic Regression and Linear SVM used TF-IDF vectorization, while deep learning models such as LSTM and BiLSTM used tokenization, sequence padding, and embedding layers.

5.1 TF-IDF Vectorization

For the Logistic Regression and Linear SVM models, comments were transformed into numerical vectors using the Term Frequency–Inverse Document Frequency (TF-IDF) technique. TF-IDF measures the importance of words in a document relative to the entire dataset. Words that appear frequently in a specific comment but less frequently across all comments receive higher importance scores.

The TF-IDF vectorizer was configured using both unigram and bigram features in order to capture individual words as well as short word combinations. Additionally, the number of extracted features was limited to the 30,000 most important features to reduce dimensionality and improve computational efficiency.

The following parameters were used:

- Maximum features: 30,000
- N-gram range: (1,2)
- Minimum document frequency: 2

- Sublinear term frequency scaling enabled

After fitting the vectorizer on the training data, the same transformation was applied to the validation and test sets.

5.2 Tokenization and Padding

For the deep learning models, text comments were converted into sequences of integers using tokenization. Each unique word was assigned a numerical index representing its position in the vocabulary.

Since neural networks require inputs of equal length, sequence padding was applied to ensure that all comments had the same size. Comments shorter than the maximum sequence length were padded with zeros, while longer comments were truncated.

The following configuration was used:

- Vocabulary size: 30,000 words
- Maximum sequence length: 120 tokens

This preprocessing step allowed the LSTM and BiLSTM models to process textual sequences consistently during training.

5.3 Word Embeddings

The deep learning models also used embedding layers to transform integer word representations into dense vector representations. Unlike TF-IDF, embeddings allow the model to learn semantic relationships between words during training. Words with similar meanings may therefore obtain similar vector representations.

An embedding dimension of 128 was used in both the LSTM and BiLSTM architectures. The embedding layer was trained together with the neural network, allowing the models to learn contextual information directly from the dataset.

6. Models Used

In this project, four different models were implemented and compared for the task of toxic comment classification. The selected models include both traditional machine learning approaches and deep learning approaches in order to analyze the strengths and limitations of each technique.

The implemented models are:

- Logistic Regression
- Linear Support Vector Machine (SVM)
- Long Short-Term Memory (LSTM)
- Bidirectional Long Short-Term Memory (BiLSTM)

The traditional machine learning models relied on TF-IDF feature extraction, while the deep learning models used sequence-based representations with embedding layers.

6.1 Logistic Regression

Logistic Regression was used as the baseline machine learning model for this project. Despite being a relatively simple algorithm, Logistic Regression is widely used in text classification tasks because of its efficiency and strong performance when combined with TF-IDF features.

The model attempts to learn the relationship between input features and output classes by estimating probabilities using a logistic function. In this project, the model was trained using TF-IDF vectors extracted from the preprocessed comments.

The following configuration was used:

- Maximum iterations: 1000
- Regularization parameter (C): 1.0
- Balanced class weights

- Random seed: 42

Class balancing was enabled to reduce the impact of dataset imbalance and improve classification performance on minority classes.

6.2 Linear Support Vector Machine (SVM)

The second traditional machine learning model implemented in this project was the Linear Support Vector Machine (SVM). SVM is a powerful classification algorithm commonly used in Natural Language Processing tasks because of its ability to handle high-dimensional feature spaces effectively.

The Linear SVM model attempts to find the optimal decision boundary that maximizes the separation margin between different classes. Similar to Logistic Regression, the model was trained using TF-IDF features generated from the textual data.

The following parameters were used:

- Linear kernel
- Regularization parameter (C): 1.0
- Balanced class weights
- Random seed: 42

The Linear SVM model generally achieved strong performance on minority classes while maintaining good overall classification accuracy.

6.3 Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) is a type of recurrent neural network designed to process sequential data while capturing long-term dependencies between words. Unlike traditional

machine learning models, LSTM networks consider word order and contextual relationships within a sentence.

In this project, the LSTM model used an embedding layer followed by an LSTM layer and fully connected dense layers for classification. The architecture was implemented using TensorFlow and Keras.

The architecture included:

- Embedding layer
- LSTM layer with 64 units
- Dense hidden layer
- Dropout regularization
- Softmax output layer

The model was trained using the Adam optimizer and sparse categorical cross-entropy loss function. Early stopping was also applied to prevent overfitting during training.

Class weights were additionally used to reduce the impact of dataset imbalance.

6.4 Bidirectional Long Short-Term Memory (BiLSTM)

The final model implemented in this project was the Bidirectional Long Short-Term Memory (BiLSTM) model. BiLSTM extends the standard LSTM architecture by processing textual sequences in both forward and backward directions. This allows the model to capture contextual information from both past and future words within a sentence.

The BiLSTM model used an embedding layer followed by a bidirectional LSTM layer and a global max-pooling layer before the dense classification layers.

The architecture included:

- Embedding layer

- Bidirectional LSTM layer
- Global max-pooling layer
- Dense hidden layer
- Dropout regularization
- Softmax output layer

Because the model analyzes context in both directions, it is generally more effective at understanding sentence structure and detecting subtle toxic language patterns. However, this improved performance comes at the cost of higher computational complexity and longer training times compared to traditional machine learning models.

7. Training Process

After preprocessing and feature engineering, the dataset was divided into training, validation, and testing sets in order to train and evaluate the models fairly. A stratified split was used to preserve the original class distribution across all subsets of the data.

The dataset was divided as follows:

- 70% training set
- 15% validation set
- 15% testing set

The training set was used to train the models, the validation set was used to monitor performance during training and reduce overfitting, while the test set was reserved for final evaluation.

7.1 Training of Traditional Machine Learning Models

The Logistic Regression and Linear SVM models were trained using TF-IDF feature vectors generated from the preprocessed comments. Both models used balanced class weights in order to reduce the impact of dataset imbalance and improve performance on minority classes.

The Logistic Regression model was trained with a maximum of 1000 iterations to ensure convergence, while the Linear SVM model used a linear decision boundary optimized for high-dimensional textual features.

The training time of both models was recorded and later compared with the deep learning models.

7.2 Training of Deep Learning Models

The LSTM and BiLSTM models were implemented using TensorFlow and Keras. The models were trained on padded token sequences generated during preprocessing.

The following training settings were used:

- Optimizer: Adam
- Loss function: Sparse categorical cross-entropy
- Batch size: 128
- Maximum epochs: 5
- Embedding dimension: 128

To reduce overfitting, dropout layers were included in both architectures. Additionally, an Early Stopping callback was applied during training. Early stopping monitored the validation loss and automatically restored the best model weights when validation performance stopped improving. Because the dataset was imbalanced, class weights were calculated and applied during training in order to penalize misclassification of minority classes more heavily.

The BiLSTM model generally required longer training time due to its more complex architecture and bidirectional processing mechanism.

7.3 Reproducibility

To ensure reproducibility of the experiments, random seeds were fixed for Python, NumPy, and TensorFlow. This helped reduce randomness during training and ensured more consistent results across multiple executions of the models.

8. Evaluation Metrics

Several evaluation metrics were used in this project to measure the performance of the implemented models. Since the dataset is imbalanced, relying only on accuracy would not provide a complete understanding of model performance. Therefore, additional metrics such as precision, recall, and F1-score were also used.

The models were evaluated on the testing set using the following metrics:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- ROC Curve and AUC

8.1 Accuracy

Accuracy measures the percentage of correctly classified comments out of the total number of comments. It is calculated using the following formula:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

- TP = True Positives
- TN = True Negatives
- FP = False Positives
- FN = False Negatives

Although accuracy is useful for measuring overall performance, it may be misleading when dealing with imbalanced datasets because the majority class can dominate the results.

8.2 Precision

Precision measures how many comments predicted as toxic or abusive were actually correct.

High precision indicates that the model produces fewer false positive predictions.

$$Precision = \frac{TP}{TP + FP}$$

Precision is important in toxicity detection because incorrectly labeling normal comments as toxic may negatively affect users.

8.3 Recall

Recall measures the ability of the model to correctly identify toxic or abusive comments. A high recall value means that fewer harmful comments are missed by the system.

$$Recall = \frac{TP}{TP + FN}$$

Recall is especially important in content moderation systems where missing harmful comments could allow toxic behavior to remain online.

8.4 F1-Score

The F1-score combines precision and recall into a single metric by calculating their harmonic mean.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

In this project, macro-averaged F1-score was considered one of the most important evaluation metrics because it gives equal importance to all classes regardless of their size in the dataset.

8.5 Confusion Matrix

A confusion matrix was generated for each model to visualize classification performance. The confusion matrix shows the number of correctly and incorrectly classified samples for each class.

This visualization helps identify which classes are more difficult to classify and where the models tend to make mistakes.

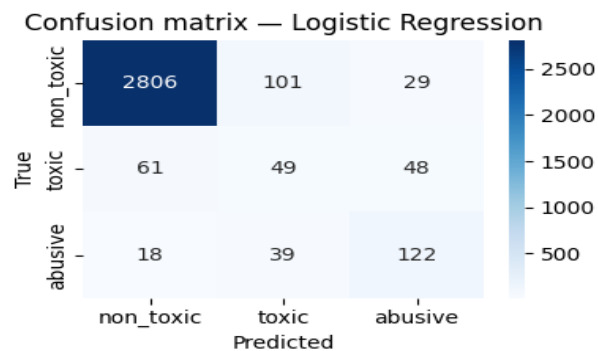


Figure 5. Confusion Matrix of the Logistic Regression Model

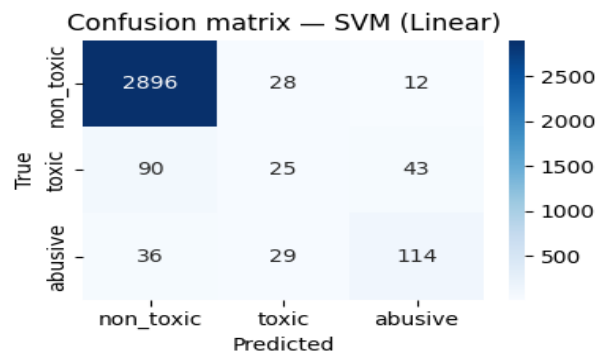


Figure 6. Confusion Matrix of the Linear SVM Model

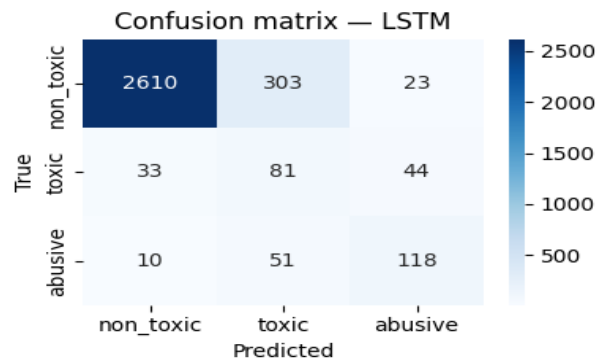


Figure 7. Confusion Matrix of the LSTM Model

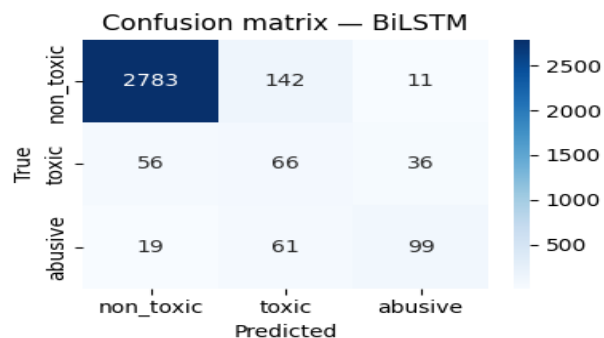


Figure 8. Confusion Matrix of the BiLSTM Model

8.6 ROC Curve and AUC

Receiver Operating Characteristic (ROC) curves were also generated to evaluate the classification capability of the models across different thresholds. The Area Under the Curve (AUC) was used to summarize overall model performance.

A higher AUC value indicates better classification performance and stronger separation between classes.

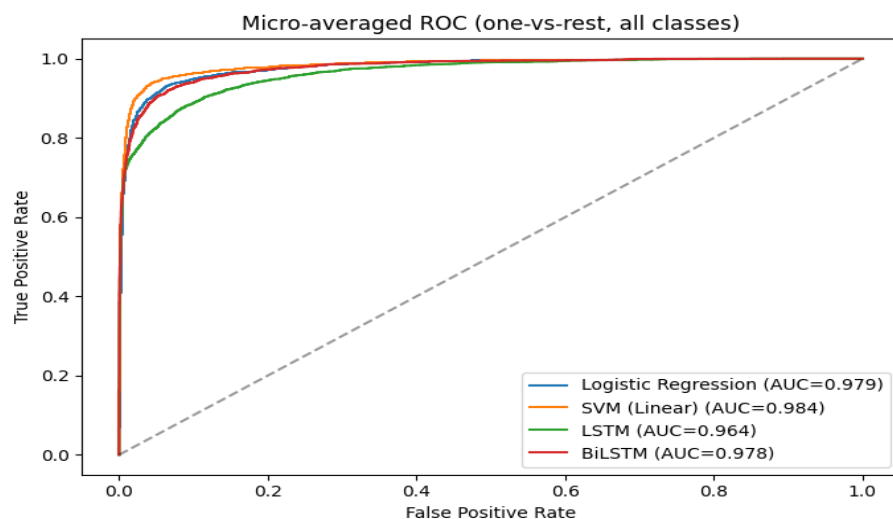


Figure 9. ROC Curve Comparison Between the Implemented Models

9. Results and Discussion

After training and evaluating all four models, their performance was compared using the evaluation metrics discussed previously. The results showed noticeable differences between traditional machine learning approaches and deep learning approaches in terms of accuracy, contextual understanding, training time, and classification performance on minority classes.

The final comparison included:

- Logistic Regression
- Linear SVM
- LSTM
- BiLSTM

The evaluation metrics used for comparison were:

- Accuracy
- Precision
- Recall
- F1-score
- Training time

Model	Accuracy	Precision	Recall	F1-Score	Training Time (s)
Logistic Regression	0.9096	0.6150	0.6491	0.6307	2.9014
Linear SVM	0.9273	0.6459	0.5938	0.6119	0.4138
LSTM	0.8582	0.6026	0.6869	0.6185	15.0799
BiLSTM	0.9007	0.6324	0.6396	0.6263	18.2856

Figure 10. Performance Comparison Between the Implemented Models

9.1 Logistic Regression Results

The Logistic Regression model achieved strong baseline performance despite being the simplest model implemented in the project. When combined with TF-IDF features, the model was able to classify non-toxic comments effectively while maintaining relatively good performance on toxic and abusive classes.

One of the main advantages of Logistic Regression was its low computational cost and fast training time. Compared to the deep learning models, it required significantly fewer resources while still producing competitive results. However, because the model relies mainly on word frequency statistics, it has limited ability to capture contextual relationships and word order within comments.

9.2 Linear SVM Results

The Linear SVM model generally improved classification performance compared to Logistic Regression, particularly for minority classes such as abusive comments. The model handled high-dimensional TF-IDF features effectively and produced strong precision scores.

The larger separation margin used by SVM helped improve class discrimination, making the model more robust for toxicity classification tasks. However, similar to Logistic Regression, the model still lacked deep contextual understanding because it relied on manually engineered textual features rather than sequential language modeling.

9.3 LSTM Results

The LSTM model demonstrated improved ability to capture contextual and sequential information within comments. Unlike traditional machine learning models, the LSTM architecture considers word order and long-term dependencies between words, allowing it to better understand sentence structure and toxic language patterns.

The model achieved stronger recall performance on toxic and abusive comments, indicating that it was more effective at identifying harmful content. However, training time increased significantly compared to the traditional machine learning models due to the complexity of neural network training.

The LSTM model also required more computational resources and careful tuning to avoid overfitting.

9.4 BiLSTM Results

Among all implemented models, the BiLSTM model generally achieved the strongest overall performance, particularly in terms of macro F1-score. By processing text sequences in both forward and backward directions, the model was able to capture richer contextual information and better understand relationships between words.

The use of bidirectional processing improved the model's ability to classify subtle toxic and abusive language patterns that may be difficult for traditional models to detect. Additionally, the global max-pooling layer helped extract the most important sequence features before classification.

Despite its improved performance, the BiLSTM model had the highest computational complexity and longest training time among all models implemented in the project.

9.5 Overall Comparison

The results of this project demonstrate that both traditional machine learning and deep learning approaches can be effective for toxicity detection tasks. Traditional models such as Logistic Regression and Linear SVM provided strong baseline performance with low computational cost, making them efficient choices for simpler classification systems.

On the other hand, deep learning models such as LSTM and BiLSTM demonstrated stronger contextual understanding and improved classification performance on minority classes. The BiLSTM model achieved the best overall balance between precision, recall, and F1-score because of its ability to analyze contextual information in both directions.

However, the improved performance of deep learning models came at the cost of increased training time, higher computational requirements, and greater architectural complexity.

10. Limitations and Future Work

Although the implemented models achieved strong performance for toxicity detection, several limitations were identified during the project. These limitations mainly relate to dataset imbalance, language complexity, and computational constraints.

One of the major challenges was the imbalance of the dataset. Non-toxic comments represented the majority of the samples, while abusive comments appeared less frequently. This imbalance can negatively affect model performance because models may become biased toward the majority class. Although class weights and macro F1-score were used to reduce this issue, minority class classification remained more difficult.

Another limitation is the complexity of natural language itself. Toxicity detection is challenging because online comments may contain sarcasm, slang, abbreviations, spelling mistakes, or indirect offensive language. Some comments may appear harmless individually but become toxic depending on context. Traditional machine learning models especially struggle to capture these deeper contextual meanings.

The deep learning models improved contextual understanding compared to traditional models; however, they required significantly longer training time and higher computational resources. Because of hardware and time limitations, the models were trained for a relatively small number of epochs. Additional tuning and larger computational resources may further improve performance.

Furthermore, the project used word-level tokenization and embeddings, which may miss important sub-word information such as misspellings or intentionally modified toxic words commonly used online.

11. Conclusion

This project presented a comparative study of machine learning and deep learning approaches for toxic comment classification using the Jigsaw Toxic Comment Classification dataset. The original multi-label dataset was transformed into a simplified three-class classification problem consisting of non-toxic, toxic, and abusive comments.

Several preprocessing and feature engineering techniques were applied to prepare the textual data for classification. Traditional machine learning models used TF-IDF vectorization, while deep learning models relied on tokenization, padding, and embedding layers to learn contextual information from textual sequences.

Four models were implemented and evaluated:

- Logistic Regression
- Linear SVM
- LSTM
- BiLSTM

The results showed that traditional machine learning models provided strong baseline performance with low computational cost and fast training time. In particular, the Linear SVM model achieved the highest accuracy among the implemented models.

Deep learning models such as LSTM and BiLSTM demonstrated stronger contextual understanding and improved ability to detect toxic and abusive language patterns. The BiLSTM model achieved strong overall performance due to its bidirectional sequence processing capabilities, although it required longer training time and higher computational resources.

Overall, the project demonstrates the effectiveness of both traditional and deep learning approaches for toxicity detection tasks. It also highlights the importance of preprocessing, feature

engineering, class balancing, and evaluation strategies when working with imbalanced Natural Language Processing datasets.

The knowledge and techniques explored throughout this project provide a strong foundation for developing more advanced automated content moderation systems in the future.